

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO5: Measure network performance using key metrics with simulation tools.
(BL4,BL5)

Assessment Tool Weightage: 30%

Annexure A

Pseudocode Writing Task (Algorithm Design)

1. In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values. The algorithm should include validation to ensure that time and total packets sent are not zero to avoid errors. It should display both the throughput and packet loss results clearly. Add logic to classify the network as Efficient if throughput is greater than 1500 bps and packet loss is less than 10%, otherwise classify it as Inefficient. Finally, ensure the algorithm outputs a complete summary of the network performance based on the calculated metrics.
2. An organization has multiple network links connecting its branches. The administrator wants to monitor bandwidth utilization and identify overloaded links. Write a pseudocode algorithm that periodically collects the bandwidth usage of each network link and stores the values in an array or list. Use a loop to continuously monitor all links. Add conditions to detect when bandwidth utilization exceeds 80% and recommend switching traffic to a backup link. Explain how this algorithm supports efficient bandwidth management and prevents network congestion.
3. A company has six branch offices connected through multiple network links. Each link has different bandwidth, delay, and utilization values, which change throughout the day. The network administrator wants to continuously identify the best communication path between the headquarters and each branch based

on overall link quality rather than just the shortest distance. Write a pseudocode algorithm that periodically collects the bandwidth, delay, and utilization of every network link and stores the values in suitable arrays or lists. Use a loop to repeat the monitoring process at regular intervals. Calculate a link quality score for each path using the collected metrics and select the path with the highest score. Include conditions to detect when a link becomes overloaded or fails, automatically selecting an alternate path and generating an alert for the administrator. Explain how your algorithm improves network reliability, routing efficiency, and fault tolerance while adapting to changing network conditions.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Question 1: Throughput & Packet Loss Calculator with Network Classification

1. Problem Understanding (Inputs, Outputs, Constraints)

- Inputs: total data received (8000 bits), time taken (4 seconds), packets sent (200), packets received (180).
- Outputs: throughput in bits per second (bps), packet loss percentage, and a network classification (Efficient/Inefficient).
- Constraint: the algorithm must validate that time and total packets sent are never zero before dividing, to avoid a divide-by-zero error.
- Constraint: classification rule is fixed — Efficient only when throughput > 1500 bps AND packet loss < 10%; otherwise Inefficient.

2. Algorithm Design Approach

- **Step 1:** Accept/declare the four input values (data bits, time, packets sent, packets received).
- **Step 2:** Validate that time_seconds and packets_sent are both non-zero; stop with an error message if not.
- **Step 3:** Compute throughput = total_data_bits / time_seconds.
- **Step 4:** Compute packets_lost = packets_sent - packets_received, then packet_loss_percentage = (packets_lost / packets_sent) × 100.
- **Step 5:** Apply the classification rule using an IF-THEN-ELSE on both computed metrics together.
- **Step 6:** Print a clearly labelled summary containing every input and computed value plus the final classification.

3. Pseudocode

```
BEGIN
    // ---- Inputs ----
    total_data_bits <- 8000
    time_seconds    <- 4
    packets_sent    <- 200
    packets_received <- 180

    // ---- Validation ----
    IF time_seconds = 0 THEN
        PRINT "Error: Time cannot be zero"
        STOP
    END IF

    IF packets_sent = 0 THEN
        PRINT "Error: Total packets sent cannot be zero"
        STOP
    END IF

    // ---- Calculations ----
    throughput <- total_data_bits / time_seconds           // bits per second
    packets_lost <- packets_sent - packets_received
    packet_loss_percentage <- (packets_lost / packets_sent) * 100
```

```

// ---- Classification ----
IF throughput > 1500 AND packet_loss_percentage < 10 THEN
    network_status <- "Efficient"
ELSE
    network_status <- "Inefficient"
END IF

// ---- Output Summary ----
PRINT "----- Network Performance Summary -----"
PRINT "Throughput      : ", throughput, " bps"
PRINT "Packets Sent     : ", packets_sent
PRINT "Packets Received  : ", packets_received
PRINT "Packet Loss      : ", packet_loss_percentage, " %"
PRINT "Network Status   : ", network_status
END

```

4. Use of Control Structures

- Two independent IF validation checks guard against division by zero for both `time_seconds` and `packets_sent` — each with its own error message and an early STOP.
- A single compound IF...AND...THEN...ELSE performs the two-condition classification (throughput AND packet loss together), avoiding nested IFs for a rule that is inherently a logical AND.
- No loop is required since the task is a one-off calculation on a fixed data set, keeping the control-flow appropriately minimal rather than over-engineered.

5. Pseudocode Structure & Readability

- Inputs, validation, calculations, classification, and output are placed in clearly commented, sequential sections so a reader can follow the algorithm top to bottom.
- Variable names (`throughput`, `packet_loss_percentage`, `network_status`) are self-descriptive, removing the need for extra explanatory comments.
- Consistent indentation inside IF blocks and a single BEGIN/END wrapper keep the structure easy to trace.

6. Completeness & Worked Example

Worked example with the given values: $\text{throughput} = 8000 / 4 = 2000$ bps (passes the >1500 test). $\text{packets_lost} = 200 - 180 = 20$, so $\text{packet_loss_percentage} = (20/200) \times 100 = 10\%$. Because the rule requires loss strictly less than 10%, and the value is exactly 10%, the classification condition fails on the packet-loss test — so the algorithm correctly reports the network as Inefficient despite the throughput being healthy, demonstrating that both conditions must hold simultaneously.

- Handles the zero-time and zero-packets-sent edge cases explicitly, rather than assuming well-formed input.
- Handles the boundary case where packet loss is exactly at the 10% threshold (shown above), which many naïve solutions miss.
- Produces a single, complete, human-readable summary rather than scattered print statements, satisfying the 'display results clearly' requirement.

Question 2: Bandwidth Monitoring & Overload Detection Across Multiple Links

1. Problem Understanding (Inputs, Outputs, Constraints)

- Inputs: a set of network links belonging to the organisation's branches, and each link's current bandwidth utilization (%), collected periodically.
- Outputs: a continuously updated store of utilization per link, and alerts/recommendations whenever a link crosses the overload threshold.
- Constraint: monitoring must be continuous/periodic (not a one-off check), since utilization changes over time.
- Constraint: the algorithm must detect overload at 80% utilization and recommend switching to a backup link at that point, not after failure.

2. Algorithm Design Approach

- **Step 1:** Maintain a list of all monitored links and a parallel array/list to store each link's most recent utilization value.
- **Step 2:** Use an outer loop that repeats the whole monitoring cycle indefinitely (or until stopped) to satisfy the 'periodically collects' requirement.
- **Step 3:** Use an inner loop to iterate over every link once per cycle and fetch its current utilization.
- **Step 4:** For each link, compare utilization against the 80% threshold and branch accordingly (alert + recommend backup, or report normal).
- **Step 5:** Pause for a fixed interval between cycles so monitoring is periodic rather than a tight, resource-wasting loop.

3. Pseudocode

```
BEGIN
  // ---- Setup ----
  DECLARE links[]           // list of link identifiers, e.g., {L1, L2, ..., Ln}
  DECLARE bandwidth_usage[] // parallel array to store % utilization per link
  THRESHOLD <- 80           // percent utilization considered overloaded
  monitoring_interval <- 60 // seconds between checks

  monitoring_active <- TRUE

  WHILE monitoring_active = TRUE DO
    FOR each link IN links DO
      bandwidth_usage[link] <- GET_CURRENT_UTILIZATION(link)

      IF bandwidth_usage[link] > THRESHOLD THEN
        PRINT "ALERT: ", link, " is overloaded at ", bandwidth_usage[link], "%"
        PRINT "Recommendation: switch traffic on ", link, " to its backup link"
        SWITCH_TRAFFIC_TO_BACKUP(link)
      ELSE
        PRINT link, " is operating normally at ", bandwidth_usage[link], "%"
      END IF
    END IF
  END WHILE
```

```
END FOR

    WAIT(monitoring_interval)    // pause before next monitoring cycle
END WHILE
END
```

4. Use of Control Structures

- An outer WHILE loop provides continuous, periodic monitoring for as long as the system is active.
- A nested FOR loop iterates over all links each cycle, so the algorithm scales automatically to any number of branch links without changing the logic.
- An IF...ELSE inside the FOR loop cleanly separates the overloaded case (alert + recommend backup) from the normal case (status report).
- A WAIT(monitoring_interval) call keeps the loop periodic and efficient rather than continuously polling with no delay.

5. Pseudocode Structure & Readability

- The two loops are clearly nested and commented, so the reader immediately sees 'repeat forever, and each time check every link'.
- Threshold and interval are declared as named constants (THRESHOLD, monitoring_interval) at the top, making the logic easy to tune without hunting through the algorithm.
- Consistent, minimal indentation keeps the FOR loop body and the IF/ELSE branches visually distinct.

6. Completeness & Worked Example

- Works for any number of links since it loops over the links array rather than hardcoding a fixed count.
- Distinguishes normal vs. overloaded links every cycle, so historical bandwidth_usage values remain available for later trend analysis if needed.
- How it supports efficient bandwidth management: continuous, threshold-based monitoring catches congestion early (at 80%, before a link saturates), and the automatic backup-link recommendation prevents packet loss and service disruption by rerouting traffic proactively rather than reactively — which is exactly how administrators prevent cascading network congestion across branch links.

Question 3: Adaptive Best-Path Selection Across Six Branch Offices Using Link Quality Score

1. Problem Understanding (Inputs, Outputs, Constraints)

- Inputs: bandwidth, delay, and utilization for every candidate link between HQ and each of the six branches, all changing throughout the day.
- Outputs: a continuously updated 'best path' per branch chosen by overall link quality (not shortest distance), plus alerts when a link is overloaded or fails.
- Constraint: selection must be based on a combined quality score from multiple metrics (bandwidth, delay, utilization), not a single factor.
- Constraint: the algorithm must be adaptive — it must re-evaluate at regular intervals and automatically switch to an alternate path on overload/failure, notifying the administrator.

2. Algorithm Design Approach

- **Step 1:** Maintain the six branches and, for each, a list of candidate links/paths from HQ.
- **Step 2:** Use an outer loop to repeat monitoring at regular intervals (adapting to changing conditions).
- **Step 3:** Use nested loops to visit every branch and every candidate link to that branch each cycle.
- **Step 4:** For each link, collect the three metrics and compute a weighted link_quality_score combining them.
- **Step 5:** Check each link for overload or failure; if detected, select and activate the best alternate link and alert the administrator.
- **Step 6:** After scoring all links for a branch, select and activate the link with the maximum score as the active path.

3. Pseudocode

```
BEGIN
  // ---- Setup ----
  DECLARE branches[] <- {B1, B2, B3, B4, B5, B6}
  DECLARE paths[branch] // list of candidate links from HQ to each branch
  DECLARE bandwidth[link], delay[link], utilization[link]
  DECLARE link_quality_score[link]
  OVERLOAD_THRESHOLD <- 80 // percent
  monitoring_interval <- 60 // seconds
  W1, W2, W3 <- weighting constants for bandwidth, delay, utilization

  monitoring_active <- TRUE

  WHILE monitoring_active = TRUE DO
    FOR each branch IN branches DO
      FOR each link IN paths[branch] DO
        bandwidth[link] <- GET_BANDWIDTH(link)
        delay[link] <- GET_DELAY(link)
        utilization[link] <- GET_UTILIZATION(link)
```

```

// Higher bandwidth is good; higher delay/utilization is bad
link_quality_score[link] <- (W1 * bandwidth[link])
                           - (W2 * delay[link])
                           - (W3 * utilization[link])

IF utilization[link] > OVERLOAD_THRESHOLD OR LINK_STATUS(link) = "DOWN" THEN
  PRINT "ALERT: link ", link, " to ", branch, " is overloaded or has failed"
  alternate_link <- SELECT_BEST_ALTERNATE(paths[branch], EXCLUDE link)
  ACTIVATE_PATH(branch, alternate_link)
  NOTIFY_ADMIN(branch, link, alternate_link)
END IF
END FOR

best_link <- LINK_WITH_MAX_SCORE(paths[branch], link_quality_score)
ACTIVATE_PATH(branch, best_link)
PRINT "Best path to ", branch, " is ", best_link,
      " with quality score ", link_quality_score[best_link]
END FOR

WAIT(monitored_interval)
END WHILE
END

```

4. Use of Control Structures

- An outer WHILE loop gives continuous, interval-based adaptation to changing network conditions.
- A nested FOR loop over branches, and within it a FOR loop over each branch's candidate links, cleanly organises the two-level (branch → link) data without duplicating logic per branch.
- A weighted-sum formula (rather than a chain of nested IFs) computes a single composite link_quality_score, making 'overall link quality' — as required by the question — directly comparable across links.
- An IF...OR condition detects both overload (utilization threshold) and outright failure (LINK_STATUS = DOWN) in one check, triggering automatic failover and admin notification.
- A separate max-score selection step (LINK_WITH_MAX_SCORE) after the inner loop chooses the best path for each branch, keeping 'scoring' and 'path selection' as distinct, readable steps.

5. Pseudocode Structure & Readability

- Two clearly nested loops (branches, then links) with consistent indentation make the two-level monitoring structure easy to follow.
- Weighting constants (W1, W2, W3) and the overload threshold are declared up front as named, tunable parameters rather than hardcoded magic numbers.
- Alerts, failover, and best-path selection are separated into distinct, well-commented blocks so each responsibility of the algorithm is easy to locate.

6. Completeness & Worked Example

- Handles the normal case (periodic re-scoring and path selection) as well as the exception cases (overload and failure) within the same monitoring cycle.
- Scales to any number of branches or candidate links per branch since both loops iterate over collections rather than fixed indices.
- How this improves reliability, routing efficiency, and fault tolerance: continuous scoring on bandwidth, delay, and utilization together (rather than static shortest-path routing) means the 'best' path always reflects current real-world conditions; the overload/failure check with automatic alternate-path activation lets the network self-heal within one monitoring interval instead of waiting for manual intervention; and admin alerts ensure a human is kept in the loop for links that need physical/ISP-level attention, giving both automated resilience and human oversight.